LAB 3

Aim: Implement an RMI client and server

Theory:

RMI stands for Remote Method Invocation. It is a mechanism that allows an object residing in one system (JVM) to access/invoke an object running on another JVM.
RMI is used to build distributed applications; it provides remote communication between Java programs. It is provided in the package java.rmi.

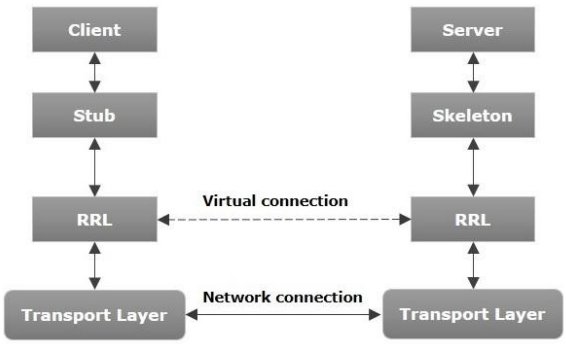
Architecture of an RMI Application

In an RMI application, we write two programs, a server program (resides on the server) and a client program (resides on the client).

- Inside the server program, a remote object is created and reference of that object is made available for the client (using the registry).
- The client program requests the remote objects on the server and tries to invoke its methods.

The following diagram shows the architecture of an RMI application.

RMI Architecture



To write an RMI Java application, you would have to follow the steps given below –

- Define the remote interface
- Develop the implementation class (remote object)
- Develop the server program
- Develop the client program
- Compile the application
- Execute the application

Code:

Calculator.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
   public int add(int a, int b) throws RemoteException;
   public int sub(int a, int b) throws RemoteException;
   public int mul(int a, int b) throws RemoteException;
   public int div(int a, int b) throws RemoteException;
}
```

CalculatorImpl.java

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;


public class CalculatorImpl extends UnicastRemoteObject implements Calculator {


   public CalculatorImpl() throws RemoteException {
      super();
   }


   public int add(int a, int b) throws RemoteException {
      return a + b;
   }


   public int sub(int a, int b) throws RemoteException {
      return a - b;
   }


   public int mul(int a, int b) throws RemoteException {
      return a * b;
   }


   public int div(int a, int b) throws RemoteException {
```

```
        if(b == 0) throw new ArithmeticException("Cannot divide by zero");
        return a / b;
    }
}
```

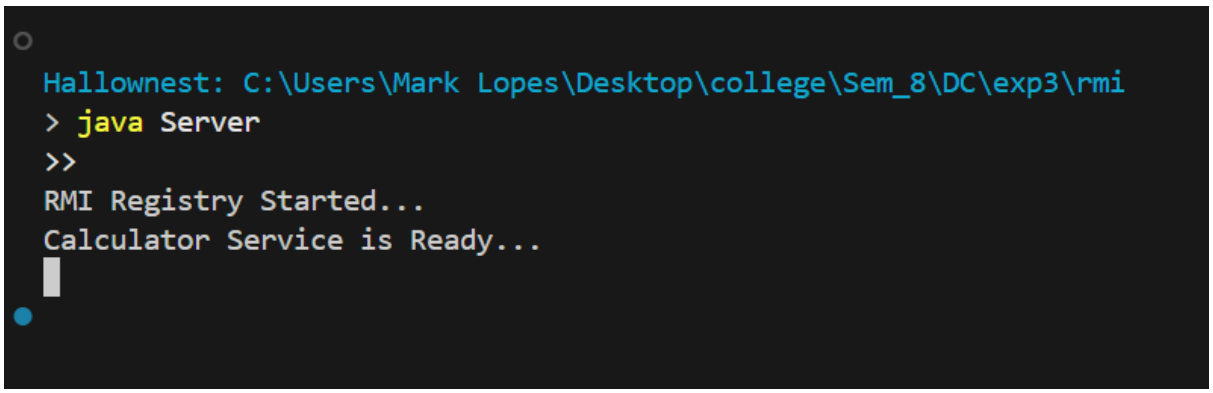
Client.java

```
import java.rmi.Naming;
import java.util.Scanner;

public class Client {
    public static void main(String[] args) {
        try {
            Calculator calc = (Calculator) Naming.lookup("rmi://localhost/calculator");
            System.out.println("Connected to Calculator Service");

            Scanner sc = new Scanner(System.in);
            System.out.print("Enter first number: ");
            int a = sc.nextInt();
            System.out.print("Enter second number: ");
            int b = sc.nextInt();

            System.out.println("Add: " + calc.add(a, b));
            System.out.println("Sub: " + calc.sub(a, b));
            System.out.println("Mul: " + calc.mul(a, b));
            System.out.println("Div: " + calc.div(a, b));

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Server.java

```
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class Server {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);
            System.out.println("RMI Registry Started...");

            CalculatorImpl calc = new CalculatorImpl();

            Naming.rebind("rmi://localhost/calculator", calc);

            System.out.println("Calculator Service is Ready...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:



```
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp3\rmi
> java Client
>>
Connected to Calculator Service
Enter first number: 10
Enter second number: 5
Add: 15
Sub: 5
Mul: 50
Div: 2
```

Conclusions:

This experiment helped us understand how Java RMI enables communication between a client and a remote server object. We implemented a distributed calculator application and learned how remote interfaces, server binding, and client lookup work. Overall, the experiment gave us practical insight into building and executing RMI-based distributed applications.

Postlab Questions:

1. What are the different times at which a client can be bound to a server?
A client can be bound to a server either at compile time (using static binding where server information is known beforehand) or at runtime (dynamic binding using the RMI registry to look up remote objects during execution).

2. How does a binding process locate a server?
The binding process locates a server through the RMI Registry, where the server registers (binds) its remote objects with a unique name. The client performs a lookup using this name (e.g., Naming.lookup("rmi://localhost/calculator")) to locate and connect to the remote server object.

3.Name some optimization methods adopted for better performance of distributed applications using RPC and RMI.
Some optimization methods include:

- Caching and memoization to reduce repeated remote calls
- Batching requests to decrease network overhead
- Using efficient serialization to speed up object transfer
- Load balancing across multiple servers